Phytium飞腾

Phytium DDMA 开发手册

(V1.1)

飞腾信息技术有限公司
www.phytium.com.cn
2023 年 08 月 18 日

版权所有 © 飞腾信息技术有限公司 2023。 保留一切权利。

未经本公司同意，任何单位、公司或个人不得擅自复制、翻译、摘抄本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。

商标声明

Phytium 和其他飞腾商标均为飞腾信息技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

特别提示

本文档仅作为使用指导，飞腾对本文档内容不做任何明示或暗示的声明或保证。本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

由于产品版本升级或其他原因，本文档内容会不定期进行更新，如有变更，恕不另行通知。

最新技术资源

飞腾嵌入式软件开源社区 https://gitee.com/phytium_embedded

或者访问飞腾软件开发者平台 https://service.phytium.com.cn/developer/

目录

1 概述 3
2 DMA Engine 框架 3
2.1 基本结构 3
2.2 源码结构 3
3 DDMA 驱动配置 4
3.1 menuconfig 配置 4
3.2 DDMA 控制器设备树配置 5
3.3 DMA 设备的设备树配置 6
4 dmaengine 接口说明 7
4.1 dma_request_chan 7
4.2 dma_release_channel 7
4.3 dmaengine_slave_config 7
4.4 dmaengine_prep_slave_sg 8
4.5 dma_async_issue_pending 9
4.6 dmaengine_terminate_all 9
4.7 dmaengine_pause 9
4.8 dmaengine_resume 9
4.9 dmaengine_tx_status 9
5 dmaengine 使用范例 10
6 FAQ 11
A 术语 11
B 更新记录 12

1 概述

本文档介绍 Phytium PE220x 系列 DDMA 驱动模块及 Linux DMA Engine 接口使用方法。DMA Engine 是 Linux 内核 dma 驱动框架，目的在于统一 DMA API，让各个模块使用 DMA 时不用关心硬件细节，同时提高代码复用。

2 DMA Engine 框架

2.1 基本结构

Linux 内核实现了 dmaengine，封装了 DMA client 驱动开发的统一接口，提出了实现 DMA 控制器驱动的接口，DMA Engine 结构关系如图 2-1：

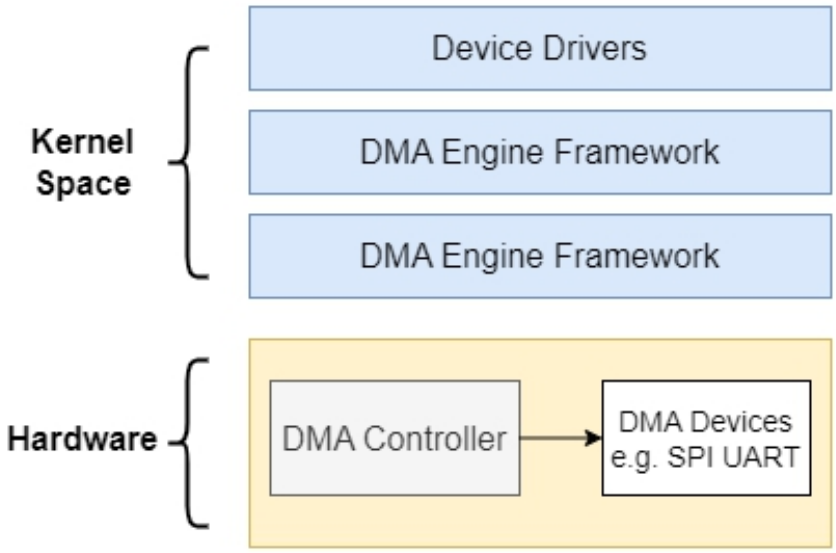

图 2- 1 DMA Engine 结构关系图

2.2 源码结构

```
linux 5.10
|
|-- drivers
|     |-- dma
|           |-- dmaengine.c
|           |-- dmaengine.h
```

```
|           |-- of-dma.c
|           |-- virt-dma.c
|           |-- virt-dma.h
|           |-- phytium
|               |-- phytium-ddmac.h
|               |-- phytium-ddmac.c
|-- include
|     |-- linux
|         |-- dmaengine.h
```

3 DDMA 驱动配置

3.1 menuconfig 配置

执行 make ARCH=arm64 meunconfig 进入配置界面，并按照以下步骤配置 DDMA 驱动。首先，选择 Device Drivers 选项进入下一级配置，如图 3-1 所示：

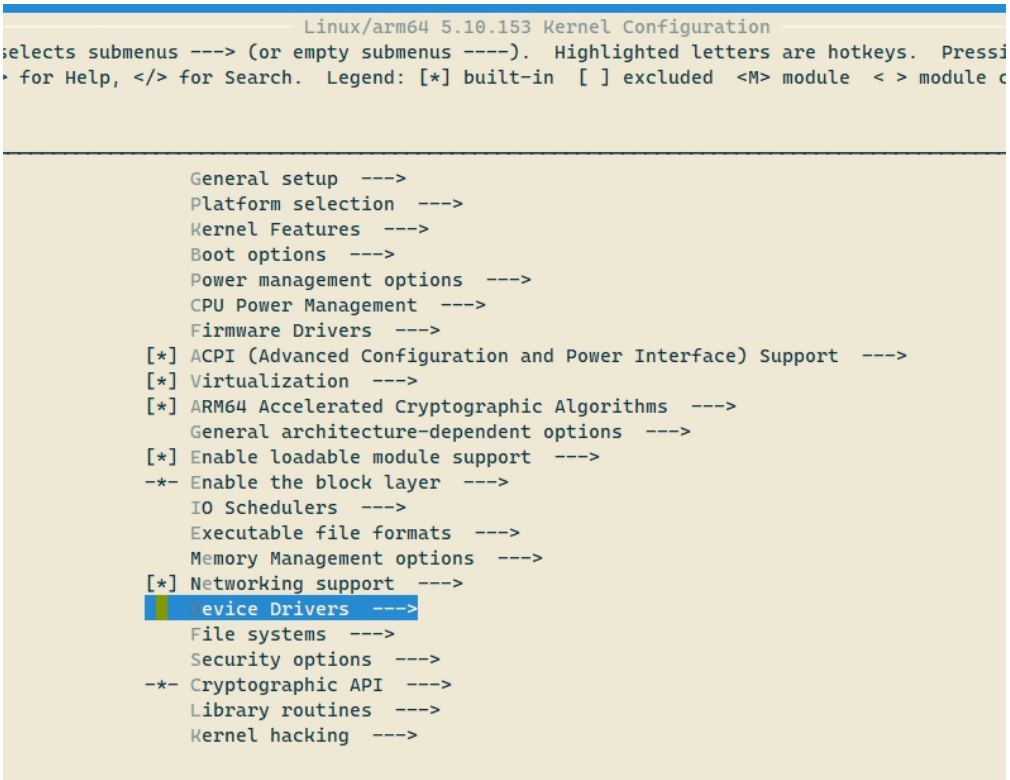

图 3- 1 menuconfig 根菜单

选择 DMA Engine support，进入下一级配置，如图 3-2 所示：

```
                                    Device Drivers
ects submenus ---> (or empty submenus ----).  Highlighted letters are hotkeys.
or Help, </> for Search.  Legend: [*] built-in  [ ] excluded  <M> module  < > m

          ^(-)
          <*> Sound card support  --->
              HID support  --->
          [*] USB support  --->
          <*> MMC/SD/SDIO card support  --->
          < > Sony MemoryStick card support  ----
          -*- LED Support  --->
          [ ] Accessibility support  ----
          < > InfiniBand support  ----
          <*> EDAC (Error Detection And Correction) reporting  --->
          [*] Real Time Clock  --->
          [*] DMA Engine support  --->
              DMABUF options  --->
          [ ] Auxiliary Display support  ----
          <M> Userspace I/O drivers  --->
          <*> VFIO Non-Privileged userspace driver framework  --->
          [ ] Virtualization drivers  ----
          [*] Virtio drivers  --->
          < > vDPA drivers  ----
          [*] VHOST drivers  --->
```

图 3– 2 Device Drivers 菜单

选择 Phytium PE220x DDMA support，如图 3-3 所示：

```
      < >   Intel Integrated DMA 64-bit support
      [*]   Marvell XOR engine version 2 support
      <*>   DMA API Driver for PL330
      < >   PLX ExpressLane PEX Switch DMA Engine Support
      < >   Xilinx AXI DMAS Engine
      < >   Xilinx ZynqMP DMA Engine
      < >   Xilinx DPDMA Engine
      [ ]   Phytium px210 PCI DMA support
      [*]   Phytium PE220x DDMA support
      < >   Qualcomm Technologies HIDMA Management support
      < >   Qualcomm Technologies HIDMA Channel support
      < >   Synopsys DesignWare AHB DMA platform driver
      < >   Synopsys DesignWare AHB DMA PCI driver
      < >   Synopsys DesignWare eDMA controller driver
      < >   Synopsys DesignWare eDMA PCIe driver
```

图 3– 3 dma driver 菜单

3.2 DDMA 控制器设备树配置

在 pe220x.dtsi 文件中，配置了 ddma 控制器的通用信息，用户一般不建议修改，由 ddma 驱动维护者

维护。

```
ddma0: ddma@28003000 {
        compatible = "phytium,ddma"; //  用于设备和驱动绑定
        reg = <0x0 0x28003000 0x0 0x1000>; // DDMA 寄存器基地址及范围
        interrupts = <GIC_SPI 75 IRQ_TYPE_LEVEL_HIGH>; //  对应的 gic 中断号
        #dma-cells = <2>; //  用于通过 dts 配置 dma，包含通道号和 dma 请求信号
        dma-channels = <8>; //  用于配置 ddma 使用的通道个数
};
```

3.3 DMA 设备的设备树配置

目前支持 uart 和 spi 使用 DDMA 进行数据传输，uart 设备仅支持发送数据使用 dma 进行传输，下面分别是 uart 和 spi 的 dma 设备树配置信息：

```
uart1: uart@2800d000 {
        compatible = "arm,pl011","arm,primecell";
        reg = <0x0 0x2800d000 0x0 0x1000>;
        interrupts = <GIC_SPI 84 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&sysclk_100mhz &sysclk_100mhz>;
        clock-names = "uartclk", "apb_pclk";
        dmas = <&ddma0 2 3>; //  对应 dma-cells，配置 ddma 通道和 dma 请求信号
        dma-names = "tx"; // dma 通道名称，驱动会根据名称获取对应的通道信息
        status = "disabled";
};

spi2: spi@2803c000 {
        compatible = "phytium,spi";
        reg = <0x0 0x2803c000 0x0 0x1000>;
        interrupts = <GIC_SPI 161 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&sysclk_50mhz>;
        num-cs = <4>;
        #address-cells = <1>;
        #size-cells = <0>;
        dmas = <&ddma0 0 8>,
                <&ddma0 1 21>;
        dma-names = "tx", "rx";
        status = "disabled";
};
```

4 dmaengine 接口说明

4.1 dma_request_chan

1） 原型：struct dma_chan *dma_request_chan(struct device *dev, const char *name)

2） 作用：申请一个和名称对应的 dma 通道

3） 参数：

dev：指向 dma 设备的指针

name：dma 通道名称，与设备树 dma-names 对应

4） 返回：申请成功返回 dma 通道指针，否则返回 NULL

4.2 dma_release_channel

1） 原型：void dma_release_channel(struct dma_chan *chan)

2） 作用：用于释放指定的 dma 通道

3） 参数：

chan：指向需要释放的 dma 通道

4.3 dmaengine_slave_config

1） 原型：int dmaengine_slave_config(struct dma_chan *chan, struct dma_slave_config *config)

2） 作用：配置 dma 通道的信息

3） 参数：

chan：指向需要操作的 dma 通道

config：dma 通道的配置信息

4） 返回：成功返回 0，失败返回错误码

dma_slave_config 的结构说明如下：

```
struct dma_slave_config {
        enum dma_transfer_direction direction; // 传输方向，MEM_TO_DEV 或者
DEV_TO_MEM
        phys_addr_t src_addr; // dma 设备数据源地址，根据传输方向来进行配置
        phys_addr_t dst_addr; // dma 设备数据目的地址，根据传输方向来进行配置
        enum dma_slave_buswidth src_addr_width; // 源数据宽度
        enum dma_slave_buswidth dst_addr_width; // 目的数据宽度
        u32 src_maxburst;
```

```
        u32 dst_maxburst;
        u32 src_port_window_size;
        u32 dst_port_window_size;
        bool device_fc;
        unsigned int slave_id;
};
```

4.4 dmaengine_prep_slave_sg

1） 原型：

struct dma_async_tx_descriptor *dmaengine_prep_slave_sg(struct dma_chan *chan, struct scatterlist *sgl, unsigned int sg_len, enum dma_transfer_direction dir, unsigend long flags, void *context)

2） 作用：准备一次 dma 传输

3） 参数：

chan：指向要操作的 dma 通道

sgl：散列表地址，将需要发送的数据以散列表形式组织

sg_len：散列表中数据表的个数

dir：数据传输的方向

flags：传输标志

4） 返回：成功返回传输描述符指针，失败返回 NULL

dma_async_tx_descriptor 传输描述符结构介绍：

```
struct dma_async_tx_descriptor {
        dma_cookie_t cookie;
        enum dma_ctrl_flags flags; /* not a 'long' to pack with cookie */
        dma_addr_t phys;
        struct dma_chan *chan;
        dma_cookie_t (*tx_submit)(struct dma_async_tx_descriptor *tx);
        int (*desc_free)(struct dma_async_tx_descriptor *tx);
        dma_async_tx_callback callback;
        dma_async_tx_callback_result callback_result;
        void *callback_param;
        ...
};
// cookie：用于唯一识别该描述符
// tx_submit：本次传输提交的执行函数
// desc_free：指向释放该描述符的函数
// callback：传输完成后的回调函数
// callback_param：回调函数传入的参数
```

4.5 dma_async_issue_pending

1） 原型：void dma_async_issue_pending(struct dma_chan *chan)

2） 作用：启动通道传输

3） 参数：

chan：指向要操作的 dma 通道

4.6 dmaengine_terminate_all

1） 原型：int dmaengine_terminate_all(struct dma_chan *chan)

2） 作用：停止通道上的所有传输

3） 参数：

chan：指向要操作的 dma 通道

4） 返回：成功返回 0，失败返回错误码

4.7 dmaengine_pause

1） 原型：int dmaengine_pause(struct dma_chan *chan)

2） 作用：暂停 dma 通道的传输

3） 参数：

chan：指向要操作的 dma 通道

4） 返回：成功返回 0，失败返回错误码

4.8 dmaengine_resume

1） 原型：int dmaengine_resume(struct dma_chan *chan)

2） 作用：恢复 dma 通道的传输

3） 参数：

chan：指向要操作的通道

4） 返回：成功返回 0，失败返回错误码

4.9 dmaengine_tx_status

原型：enum dma_status dmaengine_tx_status(struct dma_chan *chan, dma_cookie_t cookie, struct

dma_tx_state *state)

作用：查询某次传输的状态

参数：

chan：指向需要查询的 dma 通道

cookie：需要查询某次传输的 cookie id

state：用于获取传输状态

返回：

DMA_COMPLETE：表述传输已完成

DMA_IN_PROGRESS：表示提交尚未处理或处理中

DMA_PAUSED：表示传输已经暂停

DMA_ERROR：表示传输失败

5 dmaengine 使用范例

```
struct dma_chan *chan;
char *buf;
dma_cookie_t cookie;
struct dma_slave_config tx_conf = {
        .dst_addr = REG_DR,
        .dst_addr_width = DMA_SLAVE_BUSWIDTH_1_BYTE,
        .direction = DMA_MEM_TO_DEV,
        .dst_maxburst = fifosize >> 1,
        .device_fc = false,
};

// 申请 dma 通道
chan = dma_request_chan(dev, "tx");
if (IS_ERR(chan)) {
        return;
}
// 配置 dma 通道
dmaengine_slave_config(chan, &tx_conf);

// 配置 sg 散列表，用于 dma 数据传输
buf = kmalloc(PL011_DMA_BUFFER_SIZE, GFP_KERNEL | __GFP_DMA);
if (!buf) {
        return;
```

```
}
sg_init_one(dmatx->sg, buf, BUFFER_SIZE);
dmatx->sg.length = count;
if (dma_map_sg(dma_dev->dev, &dmatx->sg, 1, DMA_TO_DEVICE) != 1) {
        return -EBUSY;
}
// 准备传输
desc = dmaengine_prep_slave_sg(chan, &dmatx->sg, 1, DMA_MEM_TO_DEV,
                                DMA_PREP_INTERRUPT | DMA_CTRL_ACK);
if (!desc) {
        return -EBUSY;
}

// 设置传输完成回调函数及参数
desc->callback = tx_callback;
desc->callback_param = pararm;

// 启动 dma 传输
cookie = dmaengine_submit(desc);
dma_async_issue_pending(chan);
```

6 FAQ

1） 配置 Uart 使用 DMA 传输，启动后 dmesg 无 DMA 通道申请信息，不确定 Uart 是否正确配置为 DMA 传输?

Uart 需要在使用并进行数据传输时才会进行 DMA 通道分配。

2） 使用 Uart 进行数据收发时，DDMA 控制器中断数没有变化，但是数据收发正常，不确定 Uart 是否正确配置为 DMA 传输?

当数据量小于 PL011_DMA_BUFFER_SIZE 的一半时，默认会走 Uart 中断模式进行数据发送，可尝试增加数据发送量，观察 DDMA 控制器中断变化。

3） 同时配置 Uart 和 SPI 通过 DDMA 传输后，Uart 无法正常进行数据收发?

Uart 和 SPI 需要配置不同的 DDMA 通道，修改设备树中对应设备的 dma 通道号可以解决该问题。

A 术语

设备直接内存存取	DDMA（Device Direct Memory Access）
DMA 设备	DMA Client

DMA 通道	DMA Channel
串行外设接口	SPI
通用异步收发器	UART
散列表	Scatter List

B 更新记录

发布日期	说明
2023-06-12	初稿
2023-08-18	增加 FAQ 章节